

Curso Inicial Front-End

Clase 22: REACT: RENDERIZADO DINÁMICO Y ESTADO INICIAL

Módulo 3: Desarrollo con React.js y Ecosistema Moderno



Presentación

Las aplicaciones interactivas necesitan procesar colecciones de datos y reaccionar a las acciones del usuario. En esta clase abordaremos el Renderizado Dinámico para transformar arrays de datos en elementos visuales usando **.map()** y **key**. Además, conoceremos el concepto de Estado (**State**) a través de **useState**, el Hook más fundamental de **React**, comprendiendo cómo funciona la memoria de nuestros componentes y el flujo del renderizado.



Objetivos

- Comprender e implementar el renderizado de listas dinámicas usando **.map()** y el atributo obligatorio **key**.
- Entender el concepto de Estado (**State**), sus diferencias con las **variables** locales y por qué es inmutable.
- Dominar el uso de **useState**, comprendiendo el valor por defecto (default state) y la función despachadora (dispatch).

- Conocer y respetar las reglas absolutas en el uso de los Hooks.



Bloques Temáticos

- Tema 1: Listas, renderizado mediante `.map()` y Uso de Key props
 - El método `.map()` y el Retorno Implícito
 - La Importancia de la Prop key
 - ¿Por qué no usar `forEach`?
- Tema 2: Introducción a **useState**
 - ¿Qué son realmente los Hooks?
 - **useState**: El Avisador de React
 - Regla de Oro de React: La Inmutabilidad del Estado
 - Las Leyes Intocables de los Hooks

Un botón que no hace nada cuando lo tocas es un diseño aburrido (Estático).
Un botón que cuenta cuántas veces le has hecho clic es una aplicación real (Dinámico). Para lograr esto, un componente necesita "**Recordar**" cosas en el tiempo. Esa memoria interna en **React** se llama **Estado (State)**, y lo manejamos con una herramienta llamada Hook.

 **Recurso Oficial:** [React Docs - El Estado: Un componente con memoria](#)

Tema 1: Listas, renderizado mediante `.map()` y Uso de Key props

1. El método `.map()` y el Retorno Implícito

¿Te acuerdas cuando en las clases de JavaScript veíamos los **bucles** y como estos nos facilitaban las tareas repetitivas? Bueno en React también se utilizan por ejemplo cuando pides datos a una Base de Datos una lista de 50 usuarios, no vas a escribir 50 componentes `<TarjetaUsuario />` a mano. Vas a usar tu viejo amigo: el método de array `.map()`.

Cuando usamos `.map()`, cada iteración del array devuelve un bloque de **JSX**. Es bastante importante considerar que para que el código quede limpio, solemos utilizar paréntesis `()` en lugar de llaves `{ }` para habilitar el retorno implícito de JavaScript.

Ejemplo Visual:

```
const listaUsuarios = [  
  { id: 101, nombre: "Lucas" },  
  { id: 102, nombre: "Ana" }  
];
```

```
function Lista() {  
  return (  
    <div>  
      {listaUsuarios.map((usuario) => (  
        // Observa el uso obligatorio de 'key'  
        <TarjetaUsuario key={usuario.id} nombre={usuario.nombre} />  
      ))}  
    </div>  
  );  
}
```

2. La Importancia de la Prop key

Cuando generas una lista de elementos dinámicamente con `.map()`, **React** te obligará a pasarle una propiedad especial llamada **key** a la etiqueta raíz de cada elemento retornado. Esta prop debe ser un identificador **único** (como un **ID** de base de datos, como tú **DNI** o como la **patente** de un vehículo).

Traducción Técnica: React utiliza el algoritmo de **Reconciliación (Diffing)** para actualizar la pantalla de forma óptima. La prop **key** actúa como una "huella digital". Gracias a ella, si un elemento es agregado, modificado o eliminado, **React** no necesita re-renderizar toda la lista; *identifica exactamente cuál objeto cambió en el **DOM virtual** y aplica el parche específico en el **DOM real**.*

3. ¿Por qué no usar forEach?

- **forEach:** Se usa para recorrer arrays pero no devuelve nada (undefined). Dado que **React** necesita recibir elementos JSX válidos para poder pintarlos, `forEach` no servirá para generar interfaces.

- **.map()**: Crea y retorna un nuevo array con las transformaciones aplicadas. **React** toma ese array resultante de elementos JSX y lo inyecta directamente en el HTML. Dando como resultado varios elementos en pantalla repetidos pero con información diferente.

Tema 2: Uso de Key props e introducción a useState

1. El problema de las Variables Normales

Imagina que quieres hacer un contador de likes. Instintivamente, usarías una variable normal (`let likes = 0`) y le sumarías 1 al hacer clic.

Ejemplo del problema (Código que NO funciona visualmente):

```
function Post() {  
  let contadorLikes = 0; // Variable normal de JS  
  
  const darLike = () => {  
    contadorLikes = contadorLikes + 1;  
    console.log(contadorLikes); // En la consola, el número sí sube (1, 2,  
3...)    
  };  
  
  return (  
    <button onClick={darLike}>Likes: {contadorLikes}</button> // La  
pantalla SIEMPRE mostrará 0.  
  );  
}
```

¿Por qué fracasa? Porque cuando modificas una variable normal, el motor matemático de **JS** se entera, pero **React** es ciego. Nadie le avisa a **React** que debe "redibujar" la pantalla para reflejar ese cambio visualmente.

2. ¿Qué son realmente los Hooks?

En el pasado, para que un componente de **React** tuviera "memoria" (Estado) o pudiera ejecutar código al cargarse, estábamos obligados a usar Clases de **JavaScript**, lo cual era complejo y propenso a errores.

En 2019, **React** lanzó los **Hooks** (ganchos). Son funciones especiales que permiten que una simple función de **JavaScript** se "enganche" a las funcionalidades internas de **React**.

Analogía: Tu componente funcional es una lámpara sencilla. Por sí sola, no tiene energía. El **Hook** es el cable que conectas a la pared (el motor de **React**) para que la lámpara tenga luz (Estado) y funciones inteligentes. Sin Hooks, tus componentes serían simples plantillas estáticas de **HTML**.

En lenguaje simple: Los Hooks son funciones especiales que le dan "memoria" a tus componentes. Sin ellos, **React** se olvidaría de todo cada vez que la pantalla cambia.

Traducción Técnica: Los Hooks son funciones que permiten "enganchar" el estado y el ciclo de vida de **React** a componentes funcionales. Internamente, **React** utiliza una cola (queue) vinculada al orden de declaración para mantener la persistencia de estos datos entre renderizados.

3. **useState:** El Avisador de React

Para resolver esto, **React** nos regaló un "Hook" (un gancho o herramienta mágica) llamado **useState**. Esta herramienta hace dos cosas:

1. Crea la variable de memoria.
2. Nos entrega un "Avisador" (una función) que, al usarla, actualiza la memoria **y le grita a React que redibuje la pantalla de forma automática**.

```
import { useState } from 'react';

function PostExitoso() {
  // useState(0) significa: "El valor inicial de mi memoria será el número 0".
  // La herramienta nos devuelve SIEMPRE un Array con dos piezas, que desestructuramos:
  // 1. 'likes': La variable de lectura actual.
  // 2. 'setLikes': La función actuadora que usamos para cambiar el número y avisar a React.
  const [likes, setLikes] = useState(0);

  const darLike = () => {
    // Al usar el actuador, React actualiza el número y redibuja la pantalla al instante.
    setLikes(likes + 1);
  };
}
```

Anatomía de useState 🧠

Es el Hook más importante. Su función es permitir que el componente "recuerde" cosas.

```
const [estado, setEstado] = useState(valorInicial);
```

Desglose técnico de sus 3 partes:

1. **estado (El Valor)**: Es la variable que contiene el dato actual. Es de **solo lectura** (no puedes hacer `estado = nuevoVal`).
2. **setEstado (El Dispatch/Despachador)**: Es la única función autorizada para cambiar el valor. Cuando la llamas, le avisa a **React**: "Oye, cambié el dato, ¡redibuja la pantalla!".
3. **valorInicial**: El dato que tendrá la variable el primer milisegundo que cargue la web.

⚠ **IMPORTANTE**

Destructuring de Array: Nota que usamos corchetes `[]`. Esto es porque `useState` siempre devuelve un array de dos posiciones. Al usar corchetes, estamos "atrapando" esas dos cosas y dándoles nombre en una sola línea.

4. Regla de Oro de React: La Inmutabilidad del Estado

Nunca, bajo ningún concepto, intentes modificar la variable de estado directamente (ej: `likes++`). Siempre debes usar la función de seteo (`setLikes`).

En lenguaje simple: En **React**, si quieres cambiar un dato, no lo borras ni lo sobrescribes directamente. Tienes que crear uno nuevo y avisarle a **React** para que actualice la pantalla.

Traducción Técnica: **React** utiliza una comparación de "referencia" para saber si el estado cambió. Si modificas un objeto o array directamente, la dirección de memoria sigue siendo la misma y **React** creerá que nada cambió, ignorando el redibujado (re-render). Al usar `setEstado`, obligamos a la creación de una nueva referencia.

5. Las Leyes Intocables de los Hooks

Los Hooks como `useState` tienen magia negra por detrás, por lo que **React** te obliga a cumplir dos leyes absolutas, de lo contrario tu pantalla explotará en errores rojos:

1. **Nivel Superior:** Solo puedes llamar a un Hook en la "raíz" del componente. No puedes ocultarlos adentro de un `if`, un `for` o una función normal.

- *Razón técnica:* **React** confía en el orden de ejecución para saber cuál estado es cuál. Si pones un Hook dentro de un `if`, a veces se ejecutará y a veces no, confundiendo la memoria del motor.
2. **Solo Componentes:** No puedes usar `useState` en un archivo de **JavaScript** normal. Solo viven dentro de componentes de **React**.
 3. **Solo en Funciones de React:** No puedes usar Hooks en archivos de JavaScript puro. Solo funcionan dentro de Componentes Funcionales o de otros Hooks personalizados (Custom Hooks).
-

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **React:** <https://es.react.dev/>